GLOBAL FUSION POS: AN AUTOMATED COMPUTATIONAL ENGINE FOR MODERN HOSPITALITY

- **Core Technology:** Developed using Python 3.12 for high-speed transaction processing.
- **Industry Focus:** Specialized for the F&B (Food & Beverage) Fusion sector.
- **Presenter:** Panchami Chakraborty |VIT BHOPAL
- **Subtitle:** Bridging the gap between manual billing and complex ERP software

PROJECT VISION & SCOPE

- **Efficiency First:** Reducing the "time-to-table" for billing from minutes to seconds.
- **Accuracy:** Eliminating human error in tax and service charge calculations.
- **Accessibility:** A lightweight CLI interface that runs on any operating system.
- **Customization:** A modular menu system that adapts to seasonal food changes.
- **Financial Integrity:** Ensuring every discount and tax is applied consistently.

FINAL CODE.


```python
# 1. GLOBAL FUSION MENU DATA
# ------------------------------------------------------------
MENU = {
    "Indian Starters": {
        "1. Paneer Tikka": 340.0, "2. Chicken 65": 280.0,
        "3. Masala Papad": 85.0, "4. Veg Samosa (2pc)": 80.0,
        "5. Tandoori Prawns": 450.0, "6. Hara Bhara Kabab": 190.0,
        "7. Chicken Seekh Kabab": 320.0, "8. Fish Amritsari": 380.0
    },
    "Western Starters": {
        "9. French Fries": 120.0, "10. Garlic Bread w/ Cheese": 180.0,
        "11. Caesar Salad": 210.0, "12. Chicken Nuggets (6pc)": 220.0,
        "13. Loaded Nachos": 250.0, "14. Bruschetta": 190.0,
        "15. BBQ Chicken Wings": 310.0, "16. Mozzarella Sticks": 240.0
    },
    "Indian Main Course": {
        "17. Butter Chicken": 350.0, "18. Dal Makhani": 220.0,
        "19. Paneer Lababdar": 380.0, "20. Hyderabadi Biryani": 320.0,
        "21. Mutton Rogan Josh": 480.0, "22. Malai Kofta": 290.0,
        "23. Kadhai Chicken": 340.0, "24. Veg Jalfrezi": 220.0
    },
    "Western Main Course": {
        "25. Margherita Pizza": 290.0, "26. Veggie Burger": 210.0,
        "27. White Sauce Pasta": 280.0, "28. Grilled Sandwich": 150.0,
        "29. Fish and Chips": 420.0, "30. Pink Sauce Ravioli": 350.0,
        "31. Chicken Steak": 480.0, "32. Mushroom Risotto": 390.0
    },
    "Desserts": {
        "33. Gulab Jamun": 80.0,    "34. Chocolate Brownie": 160.0,
        "35. Apple Pie": 180.0,     "36. New York Cheesecake": 240.0,
        "37. Rasmalai (2pc)": 100.0, "38. Tiramisu": 200.0,
        "39. Ice Cream Scoop": 70.0, "40. Gajar Halwa": 120.0
    },
    "Beverages": {
        "41. Masala Chai": 30.0,    "42. Coca Cola/Pepsi": 60.0,
        "43. Cold Coffee": 110.0,   "44. Fresh Lime Soda": 80.0,
        "45. Virgin Mojito": 140.0, "46. Oreo Milkshake": 180.0,
        "47. Hot Chocolate": 130.0, "48. Mineral Water": 40.0
    }
}

# Financial Rules
GST_RATE = 0.05            # 5% GST
SERVICE_CHARGE = 0.10      # 10% Service Charge
DISCOUNT_THRESHOLD = 2000  # Bills above 2k get a discount
DISCOUNT_RATE = 0.10       # 10% Discount
```

```python
# ------------------------------------------------------------
# 2. INTERFACE & SEARCH LOGIC
# ------------------------------------------------------------
def show_pos_menu():
    print("\n" + "╔" + "═"*55 + "╗")
    print("╚" + "═"*55 + "╝")

    for category, items in MENU.items():
        print(f"\n>> {category.upper()}")
        print("-" * 57)
        for item, price in items.items():
            print(f"  {item:45} ₹{price:>8.2f}")
    print("\n" + "═"*57)

def get_menu_item(item_id):
    """Search for item by ID number (1-48)."""
    for category in MENU.values():
        for full_name, price in category.items():
            if full_name.startswith(str(item_id) + "."):
                name = full_name.split(". ")[1]
                return name, price
    return None, None

# ------------------------------------------------------------
# 3. TRANSACTION ENGINE
# ------------------------------------------------------------
def start_transaction():
    current_bill = []
    show_pos_menu()

    print("\n[CONTROLS]: Enter Item # | '0' to Generate Bill | '99' to Clear")

    while True:
        entry = input("\nSelect Item ID: ").strip()

        if entry == '0':
            break
        elif entry == '99':
            current_bill = []
            print("(!) Cart Cleared.")
            continue

        try:
            choice = int(entry)
            name, price = get_menu_item(choice)
```

```python
# ------------------------------------------------------------
def start_transaction():
    current_bill = []
    show_pos_menu()

    print("\n[CONTROLS]: Enter Item # | '0' to Generate Bill | '99' to Clear")

    while True:
        entry = input("\nSelect Item ID: ").strip()

        if entry == '0':
            break
        elif entry == '99':
            current_bill = []
            print("(!) Cart Cleared.")
            continue

        try:
            choice = int(entry)
            name, price = get_menu_item(choice)

            if name:
                qty_str = input(f"   Enter quantity for [{name}]: ")
                qty = int(qty_str) if qty_str.isdigit() and int(qty_str) > 0 else 1

                # Check for existing items in bill to update quantity
                found = False
                for item in current_bill:
                    if item['name'] == name:
                        item['qty'] += qty
                        item['total'] = item['qty'] * item['unit']
                        found = True
                        break

                if not found:
                    current_bill.append({
                        "name": name, "unit": price,
                        "qty": qty, "total": price * qty
                    })
                print(f"   (+) Added {qty} {name}(s) to bill.")
            else:
                print("   (!) Invalid ID. Refer to menu numbers (1-48).")
        except ValueError:
            print("   (!) Input must be a number.")

    if current_bill:
        finalize_invoice(current_bill)
```

```python
                if not found:
                    current_bill.append({
                        "name": name, "unit": price,
                        "qty": qty, "total": price * qty
                    })
                print(f"   (+) Added {qty} {name}(s) to bill.")
            else:
                print("   (!) Invalid ID. Refer to menu numbers (1-48).")
        except ValueError:
            print("   (!) Input must be a number.")

    if current_bill:
        finalize_invoice(current_bill)
    else:
        print("Empty bill. Session closed.")

# ------------------------------------------------------------
# 4. FINAL INVOICE GENERATION
# ------------------------------------------------------------
def finalize_invoice(items):
    subtotal = sum(i['total'] for i in items)

    # Calculate Discounts
    discount_val = 0
    if subtotal > DISCOUNT_THRESHOLD:
        discount_val = subtotal * DISCOUNT_RATE

    net_after_discount = subtotal - discount_val
    gst_val = net_after_discount * GST_RATE
    service_val = net_after_discount * SERVICE_CHARGE
    grand_total = net_after_discount + gst_val + service_val

    print("\n\n" + "═"*57)
    print(" " * 18 + "OFFICIAL TAX INVOICE")
    print("═"*57)
    print(f"{'DESCRIPTION':28} {'QTY':>4} {'PRICE':>10} {'TOTAL':>10}")
    print("-" * 57)

    for i in items:
        print(f"{i['name']:28} {i['qty']:>4} {i['unit']:>10.2f} {i['total']:>10.2f}")

    print("-" * 57)
    print(f"{'ITEM TOTAL (SUBTOTAL):':43} ₹{subtotal:>10.2f}")

    if discount_val > 0:
        print(f"{'LOYALTY DISCOUNT (10%):':43} -₹{discount_val:>9.2f}")
```

```python
    print("-" * 57)
    print(f"{'ITEM TOTAL (SUBTOTAL):':43} ₹{subtotal:>10.2f}")

    if discount_val > 0:
        print(f"{'LOYALTY DISCOUNT (10%):':43} -₹{discount_val:>9.2f}")
        print(f"{'NET AMOUNT:':43} ₹{net_after_discount:>10.2f}")

    print(f"{'GST (5%):':43} ₹{gst_val:>10.2f}")
    print(f"{'SERVICE CHARGE (10%):':43} ₹{service_val:>10.2f}")
    print("=" * 57)
    print(f"{'GRAND TOTAL (ROUNDED):':43} ₹{round(grand_total):>10}")
    print("=" * 57)
    print("\n      Thank you for choosing Global Fusion!      ")
    print("          Please visit us again soon.            ")
    print("═"*57 + "\n")

if __name__ == "__main__":
    start_transaction()
```

INPUTS AND THE FINAL OUTPUT OF THE CODE

```
GLOBAL FUSION RESTAURANT
POS BILLING SYSTEM

INDIAN STARTERS
-------------------------------------------------------
1. Paneer Tikka                    ₹  240.00
2. Chicken 65                      ₹  280.00
3. Masala Papad                    ₹   45.00
4. Veg Samosa (2pc)                ₹   60.00
5. Tandoori Prawns                 ₹  450.00
6. Hara Bhara Kabab                ₹  190.00
7. Chicken Seekh Kabab             ₹  320.00
8. Fish Amritsari                  ₹  380.00

WESTERN STARTERS
-------------------------------------------------------
9. French Fries                    ₹  120.00
10. Garlic Bread w/ Cheese         ₹  180.00
11. Caesar Salad                   ₹  210.00
12. Chicken Nuggets (6pc)          ₹  220.00
13. Loaded Nachos                  ₹  250.00
14. Bruschetta                     ₹  190.00
15. BBQ Chicken Wings              ₹  310.00
16. Mozzarella Sticks              ₹  240.00

INDIAN MAIN COURSE
-------------------------------------------------------
17. Butter Chicken                 ₹  350.00
18. Dal Makhani                    ₹  220.00
19. Paneer Lababdar                ₹  260.00
20. Hyderabadi Biryani             ₹  320.00
21. Mutton Rogan Josh              ₹  480.00
22. Malai Kofta                    ₹  290.00
23. Kadhai Chicken                 ₹  340.00
24. Veg Jalfrezi                   ₹  230.00

WESTERN MAIN COURSE
-------------------------------------------------------
25. Margherita Pizza               ₹  290.00
26. Veggie Burger                  ₹  210.00
27. White Sauce Pasta              ₹  280.00
28. Grilled Sandwich               ₹  150.00
29. Fish and Chips                 ₹  420.00
30. Pink Sauce Ravioli             ₹  350.00
31. Chicken Steak                  ₹  480.00
32. Mushroom Risotto               ₹  390.00

DESSERTS
-------------------------------------------------------
33. Gulab Jamun                    ₹   80.00
34. Chocolate Brownie              ₹  160.00
```

```
>> WESTERN MAIN COURSE
-------------------------------------------------------
  25. Margherita Pizza             ₹  290.00
  26. Veggie Burger                ₹  210.00
  27. White Sauce Pasta            ₹  280.00
  28. Grilled Sandwich             ₹  150.00
  29. Fish and Chips               ₹  420.00
  30. Pink Sauce Ravioli           ₹  350.00
  31. Chicken Steak                ₹  480.00
  32. Mushroom Risotto             ₹  390.00

>> DESSERTS
-------------------------------------------------------
  33. Gulab Jamun                  ₹   80.00
  34. Chocolate Brownie            ₹  160.00
  35. Apple Pie                    ₹  190.00
  36. New York Cheesecake          ₹  240.00
  37. Rasmalai (2pc)               ₹  100.00
  38. Tiramisu                     ₹  280.00
  39. Ice Cream Scoop              ₹   70.00
  40. Gajar Halwa                  ₹  120.00

>> BEVERAGES
-------------------------------------------------------
  41. Masala Chai                  ₹   30.00
  42. Coca Cola/Pepsi              ₹   60.00
  43. Cold Coffee                  ₹  110.00
  44. Fresh Lime Soda              ₹   80.00
  45. Virgin Mojito                ₹  140.00
  46. Oreo Milkshake               ₹  180.00
  47. Hot Chocolate                ₹  130.00
  48. Mineral Water                ₹   40.00

[CONTROLS]: Enter Item # | '0' to Generate Bill | '99' to Clear

Select Item ID: 33
   Enter quantity for [Gulab Jamun]: 2
   (+) Added 2 Gulab Jamun(s) to bill.

Select Item ID: 24
   Enter quantity for [Veg Jalfrezi]: 2
   (+) Added 2 Veg Jalfrezi(s) to bill.

Select Item ID: 11
   Enter quantity for [Caesar Salad]: 2
   (+) Added 2 Caesar Salad(s) to bill.

Select Item ID: 0
```

```
OFFICIAL TAX INVOICE

DESCRIPTION                QTY      PRICE      TOTAL
-------------------------------------------------------
Gulab Jamun                  2      80.00     160.00
Veg Jalfrezi                 2     230.00     460.00
Caesar Salad                 2     210.00     420.00
-------------------------------------------------------
ITEM TOTAL (SUBTOTAL):                  ₹   1040.00
GST (5%):                               ₹     52.00
SERVICE CHARGE (10%):                   ₹    104.00
=======================================================
GRAND TOTAL (ROUNDED):                  ₹      1196
=======================================================

      Thank you for choosing Global Fusion!
          Please visit us again soon.

PS C:\aiml project>
```

THE "PAPER-TO-DIGITAL" PROBLEM

- **Revenue Leakage:** Identifying how manual math errors lead to significant monthly losses.
- **Staff Burnout:** Reducing the mental load on servers during peak "rush hour" periods.
- **Paper Waste:** Moving away from handwritten KOTs (Kitchen Order Tickets) and bills.
- **Audit Trails:** Solving the problem of untraceable transactions and "missing" cash.
- **Customer Experience:** Eliminating long wait times for final bill generation.

KEY FUNCTIONAL REQUIREMENTS

- **Multi-Category Navigation:** Distinct sections for Indian, Western, and Fusion items.
- **ID-Based Input:** Using unique numerical codes to minimize typing for staff.
- **Dynamic Cart Logic:** Real-time adding, updating, and clearing of ordered items.
- **Automated Tax Engine:** Hardcoded GST and Service Charge rules for compliance.
- **Loyalty Incentives:** Built-in logic to trigger discounts for high-value customers

SYSTEM ARCHITECTURE & DATA FLOW

- **Modular Design:** Separating Menu Data, Transaction Logic, and Invoice Rendering.
- **Input Layer:** Captures User IDs and quantities via the command-line interface.
- **Validation Layer:** Sanitizes inputs to ensure only numbers and valid IDs pass through.
- **Processing Layer:** Calculates row totals and manages the temporary "Session Cart."
- **Output Layer:** Generates a professional, formatted ASCII-based Tax Invoice.

DATABASE STRATEGY: NESTED DICTIONARIES

- **Key-Value Pairs:** Utilizing Python Dictionaries for high-performance data retrieval.
- **Categorization:** Grouping items logically (e.g., "Indian Main Course") for easier management.
- **Price Mapping:** Floating-point precision for accurate currency representation.
- **Index Parsing:** Splitting keys (e.g., "1. Paneer Tikka") to enable ID-based searching.
- **Easy Maintenance:** Non-technical owners can update prices directly in the script.

THE SELECTION LOGIC (ID MAPPING)

- **Loop-Search Algorithm:** Iterating through nested categories to find specific Item IDs.
- **String Parsing:** Using .startswith() and .split() to identify the correct menu item.
- **Return Values:** Returning both the item name and unit price for calculation.
- **Null Handling:** Returning None if an ID is not found, preventing system crashes.
- **Speed:** Instantaneous lookup even with a menu of 100+ items.

REAL-TIME TRANSACTION MANAGEMENT

- **The "Cart" Object:** A dynamic list that stores dictionaries of active orders.
- **Duplicate Detection:** Scanning the cart to see if an item already exists.
- **Quantity Accumulation:** Adding new quantities to existing items rather than duplicating lines.
- **Auto-Totaling:** Recalculating the row total whenever a quantity change occurs.
- **Session Isolation:** Each transaction is fresh, ensuring no data bleed between tables.

USER INTERFACE DESIGN (CLI AESTHETICS)

- **Visual Framing:** Using double-line borders (═) for a premium "bill" look.
- **Column Alignment:** Using f-string padding (:28) to create a perfectly aligned table.
- **Emphasis:** Bolding and capitalizing headers to guide the user's eye.
- **Currency Symbols:** Integrating the "₹" symbol for localized financial clarity.
- **Action Cues:** Clear prompts like [CONTROLS]: Enter Item # for staff guidance.

FINANCIAL LOGIC – SUBTOTAL & DISCOUNTS

- **Base Summation:** Aggregating all row totals into a single "Subtotal" variable.
- **Threshold Logic:** Checking if the bill exceeds the ₹2,000 "Loyalty" limit.
- **Rate Application:** Calculating the 10% discount value based on the subtotal.
- **Net Amount:** Subtracting the discount *before* tax application to ensure fairness.
- **Transparency:** Showing the discount as a negative line item on the final receipt

FINAL TIPS & TAKEAWAYS

- **Service Charge:** Applying a 10% hospitality fee to the net amount.
- **The Formula:** $Grand Total = Net + (Net \times 0.05) + (Net \times 0.10)$.
- **Financial Accuracy:** Using 2-decimal point precision for all intermediate math.
- **GST Calculation:** Applying a fixed 5% tax on the net (after-discount) amount.
- **Rounding:** Using the round() function for the final payment amount.

ERROR HANDLING & ROBUSTNESS

- **Integer Casting:** Using try...except to catch errors when text is entered instead of numbe
- **Quantity Guards:** Ensuring quantities are positive integers (cannot order -1 items).
- **Range Checks:** Validating that IDs fall strictly between 1 and 48.
- **Safe Exit:** Providing a dedicated command ('0') to finish the transaction safely.
- **Cart Wipe:** Allowing '99' to clear all mistakes instantly without restarting the app.

TECHNICAL DEEP DIVE: THE LOOP ENGINE

- **Infinite Loop Pattern:** Using while True to maintain the interactive POS session.
- **Break Conditions:** Terminating the loop only when the "Generate Bill" signal is received.
- **State Management:** Variables are updated in-memory for zero-latency performance.
- **Interactive Input:** The input() function pauses the engine to wait for staff action.
- **Flow Control:** Using continue to skip invalid logic and keep the app running.

THE OUTPUT: ANATOMY OF A TAX INVOICE

- **Branding Header:** A high-impact blockquote section for restaurant identity.
- **Itemized Rows:** Detailed breakdown of Description, Qty, Price, and Total.
- **Separator Lines:** Using dashed lines to distinguish between order list and tot
- **Financial Summary:** Clear display of Subtotal, Net, GST, and Service Charg
- **Call to Action:** A friendly footer message to encourage repeat visits.

PERFORMANCE & EFFICIENCY

- **Zero Dependencies:** Runs on standard Python libraries with no external installs.
- **Low RAM Usage:** The entire system consumes less than 20MB of memory.
- **Fast Execution:** Capable of processing hundreds of orders per second.
- **Cross-Platform:** Works seamlessly on Windows, macOS, and Linux terminals.
- **Portable Code:** A single .py file that can be shared or moved easily.

UX CONSIDERATIONS FOR STAFF

- **Minimal Cognitive Load:** No need to memorize prices; only menu numbers.
- **Confirmation Feedback:** Immediate "Added" messages prevent double-entry errors.
- **Logical Categories:** Grouping items by cuisine type matches the flow of a typical meal.
- **No Mouse Required:** 100% keyboard-driven for maximum speed during busy hours.
- **Large Totals:** Final Grand Total is the most prominent part of the output.

SUMMARY & FINAL IMPACT

- **Scalable Solution:** Can grow from a single-counter cafe to a busy restaurant.
- **Operational Control:** Gives management full visibility over pricing and taxes.
- **Modernization:** Elevates the restaurant's image through professional billing.
- **Profitability:** Directly improves the bottom line by eliminating manual errors.
- **Open Floor:** Inviting questions on the code structure or implementation

THANK YOU

PANCHAMI CHAKRABORTY
panchami.25bai10587@vitbhopal.ac.in